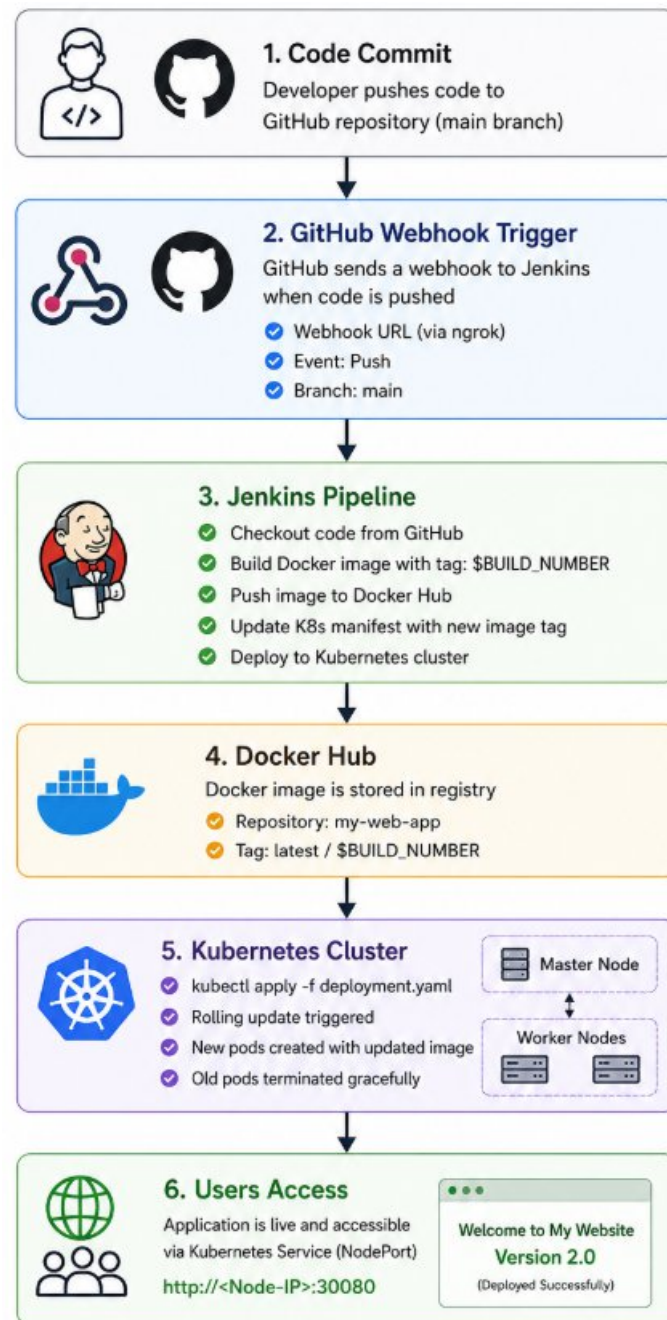


End-to-End Automated CI/CD Pipeline for Web Application Deployment on Kubernetes



This project demonstrates a modern DevOps workflow, ensuring zero-downtime updates and secure credential management across distributed virtual machines.

Introduction & System Architecture

The pipeline is orchestrated across distinct environments:

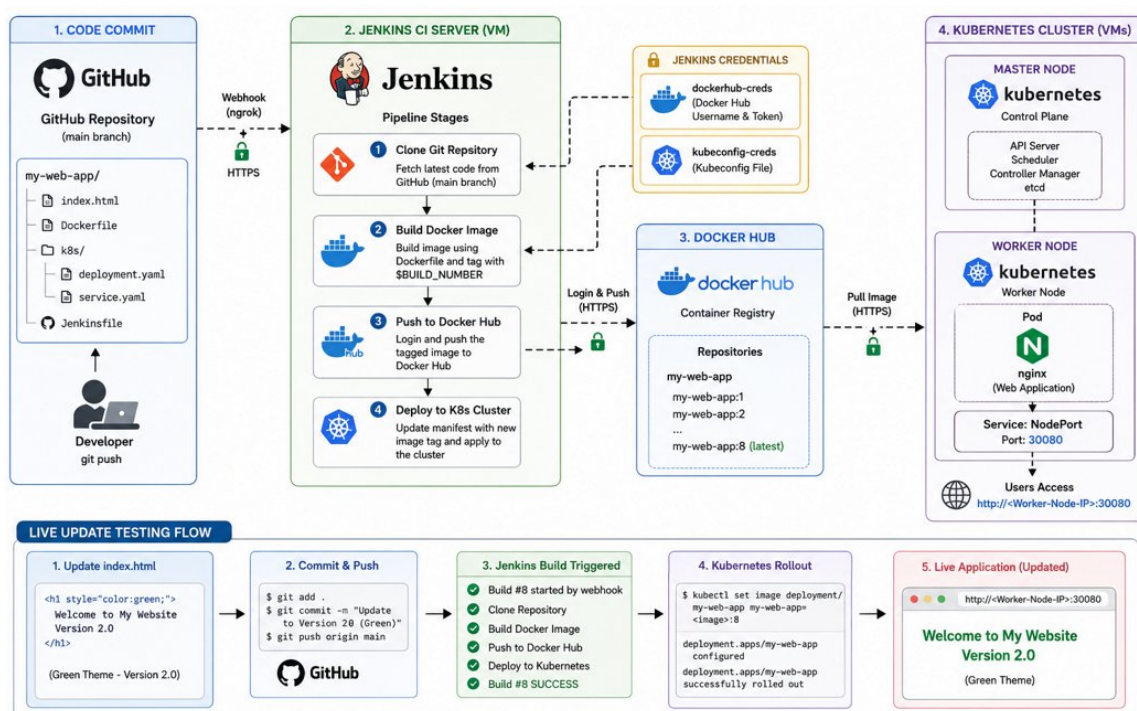
Code Commit: Pushed application code (HTML/CSS) to the main branch on GitHub.

Continuous Integration (Jenkins): A GitHub webhook detects the push and triggers a Jenkins pipeline running on a dedicated VM.

Container Registry (Docker Hub): Jenkins securely authenticates with Docker Hub, builds a new Docker image from the provided Dockerfile, tags it dynamically with the Jenkins build number, and pushes the image.

Continuous Deployment (Kubernetes): Jenkins executes remote kubectl commands to inject the new image tag into the Kubernetes deployment.yaml and applies the rollout to the cluster.

User Access: The application is live and accessible via a Kubernetes NodePort.



Prerequisites & Infrastructure Setup

Before building the pipeline, the following infrastructure was provisioned:

GitHub Repository: Hosting the application source code and deployment manifests.

Docker Hub Account: To act as the centralized container registry.

Jenkins VM: A Linux virtual machine configured with Docker, kubectl, and ngrok for webhook routing.

Kubernetes Cluster: A multi-VM bare-metal setup (Master + Worker nodes) for container orchestration.

Implementation Steps

Step 1: Preparing the Codebase & Manifests

Declarative infrastructure-as-code repository:

```
[root@localhost kube_jenkins_pipeline]# tree
```

```
.
├── Dockerfile
├── index.html
├── Jenkinsfile
└── k8s
    ├── deployment.yaml
    └── service.yaml
```

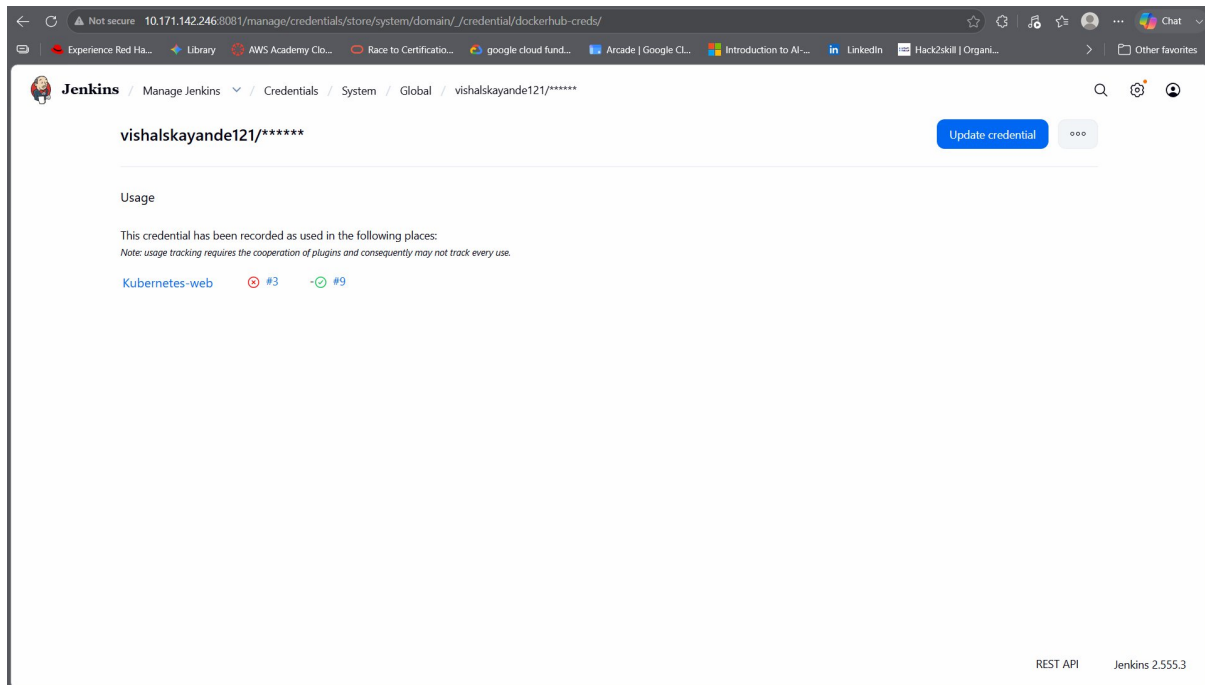
```
1 directory, 5 files
```

```
[root@localhost kube_jenkins_pipeline]#
```

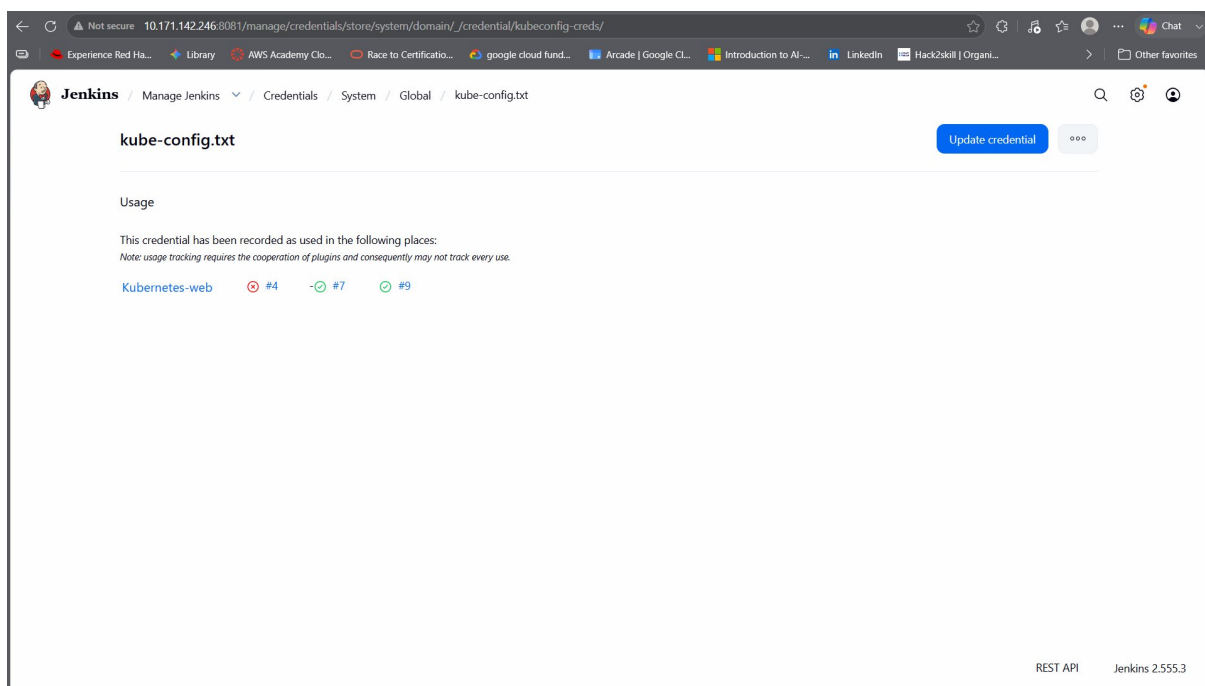
Step 2: Securing Pipeline Secrets

To ensure personal details and access tokens were never exposed in version control, I utilized Jenkins Global Credentials:

Injected Docker Hub authentication tokens (dockerhub-creds).



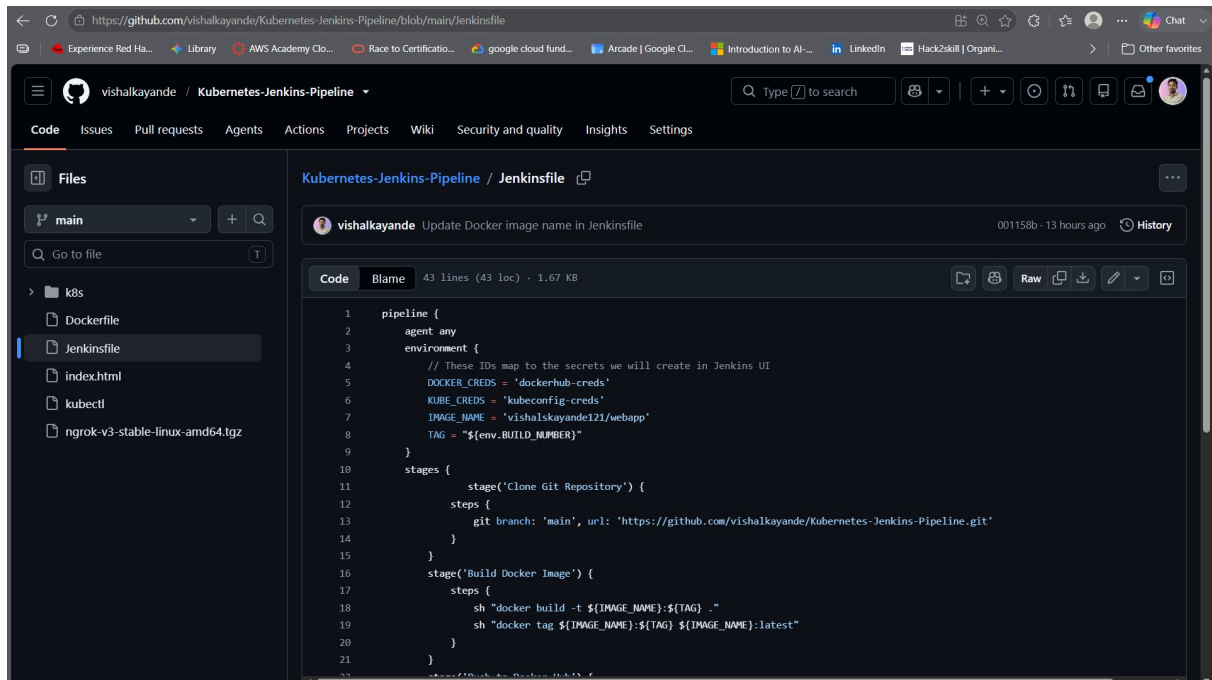
Securely stored the Kubernetes cluster configuration file (kubeconfig-creds) inside the Jenkins vault.



Step 3: Developing the Jenkinsfile

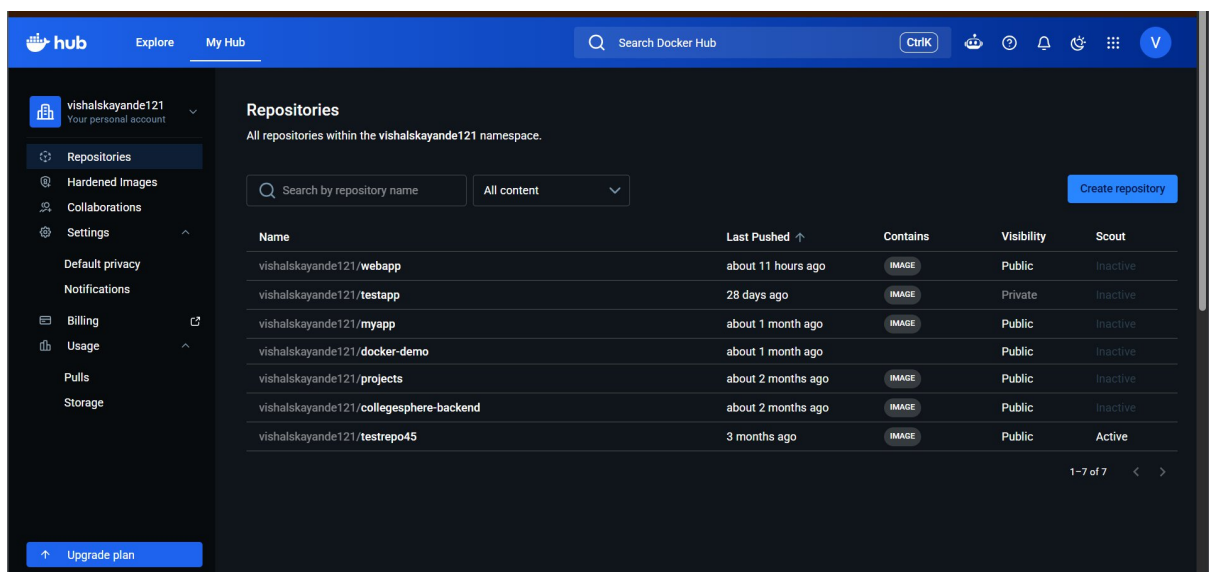
Authored a multi-stage, declarative Jenkinsfile to automate the workflow. The script executes the following stages:

Clone Git Repository: Fetches the latest commit from the main branch.



Build Docker Image: Compiles the container and applies the \$BUILD_NUMBER tag.

Push to Docker Hub: Uses masked credentials to log in and push the versioned image.



Deploy to K8s Cluster: Authenticates with the Kubernetes master node, updates the manifest with the new image tag using sed, and applies the changes via kubectl.

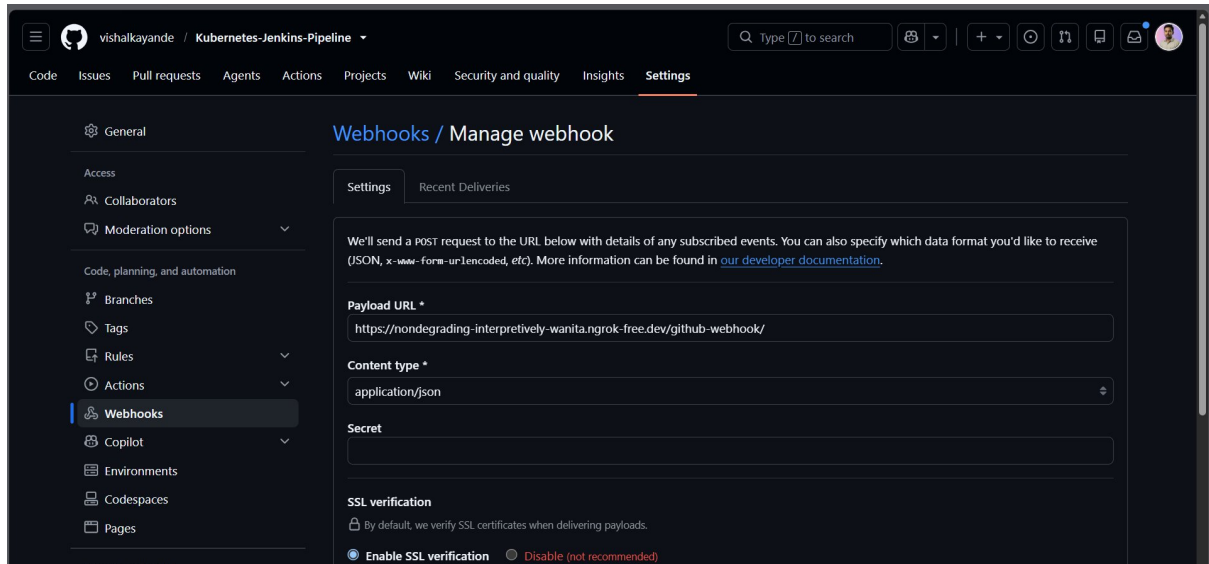
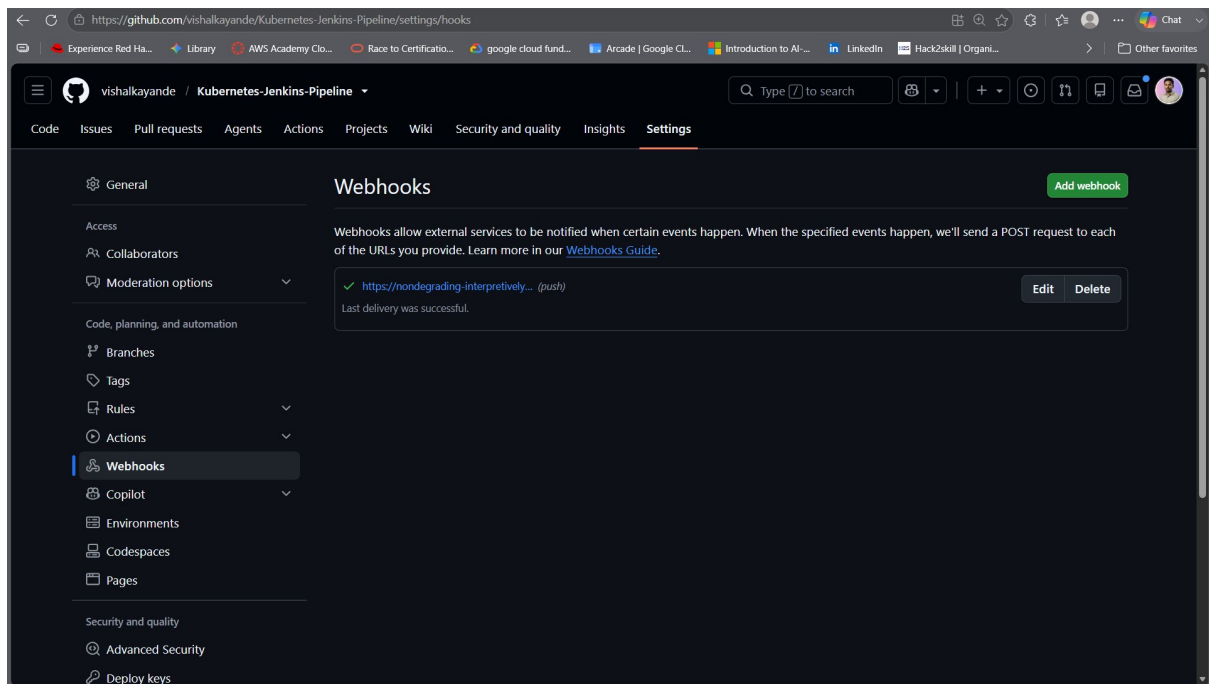
```

root@vishal:~# kubectl get pod -w
NAME          READY   STATUS    RESTARTS   AGE
my-httpd      1/1     Running   1 (62m ago) 10h
webapp-deployment-77bdd9c759-55dbj  1/1     Running   0           3m7s
webapp-deployment-77bdd9c759-bkltv  1/1     Running   0           3m7s

```

Step 4: Webhook Integration

To achieve true continuous deployment, I established an automated webhook between GitHub and Jenkins. Using ngrok to securely tunnel internet traffic to my local Jenkins VM, any git push instantly triggers the pipeline sequence.



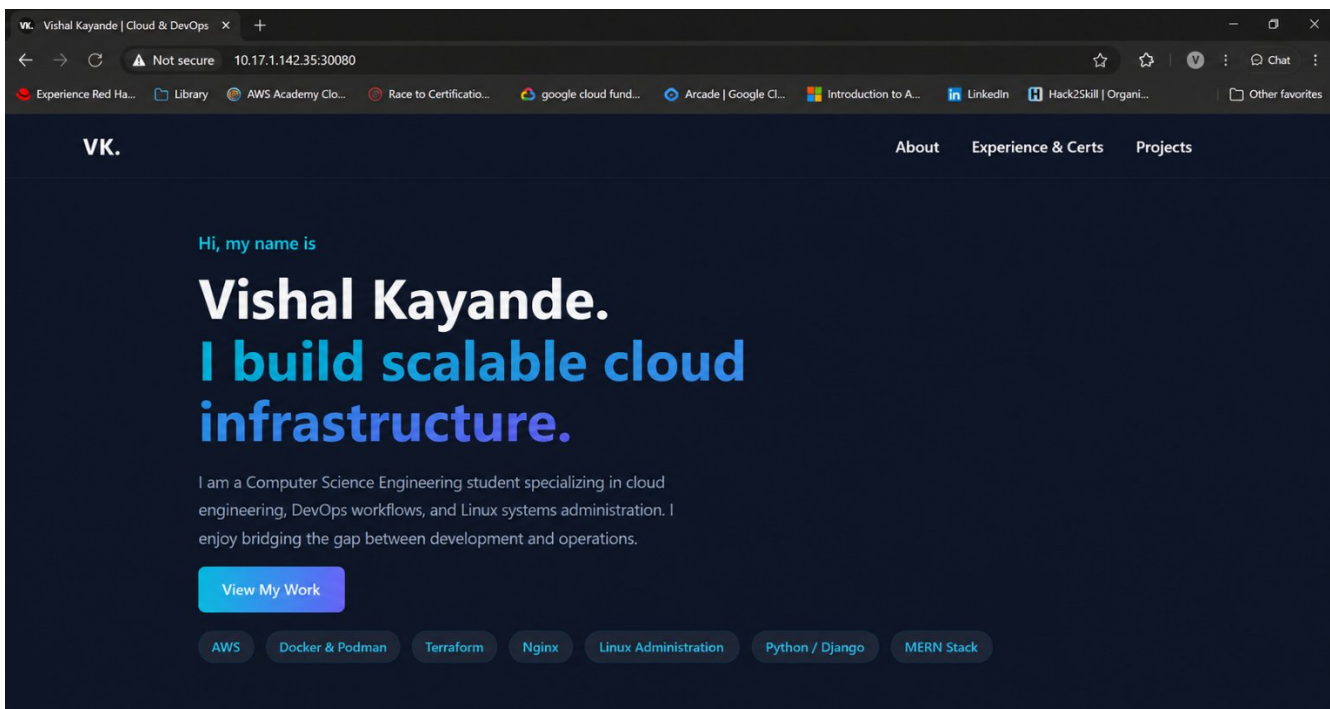
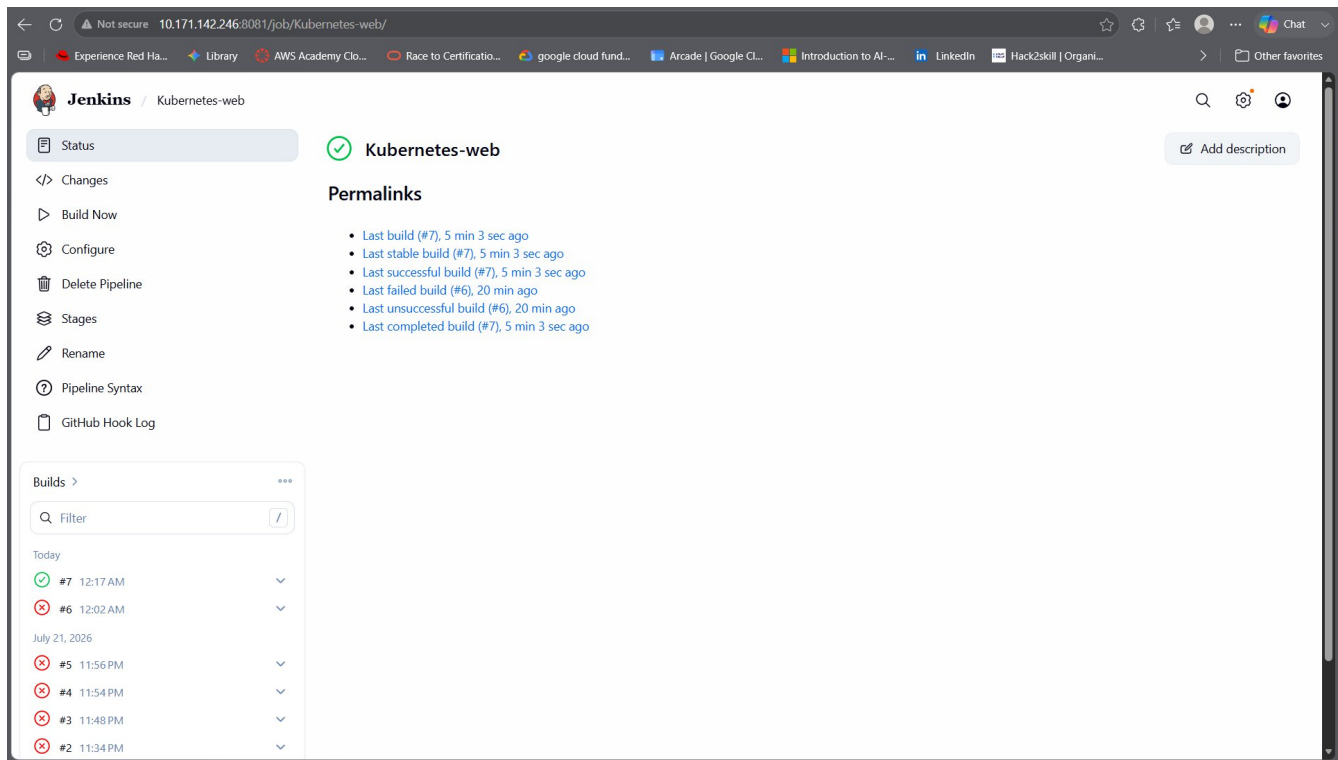
Kubernetes cluster Working:

```

root@vishal:~# kubectl get pod -w
NAME          READY   STATUS    RESTARTS   AGE
my-httpd      1/1     Running   1 (62m ago) 10h
webapp-deployment-77bdd9c759-55dbj  1/1     Running   0           3m7s
webapp-deployment-77bdd9c759-bkltv  1/1     Running   0           3m7s

```

Job Triggers automatically after commit changes to Code and Pushed on Git:



Step 5: Verification & Live Update Testing:

To verify the end-to-end automation, executed a live update:

Modified the index.html file to transition the application to a "Version 2.0" green theme.

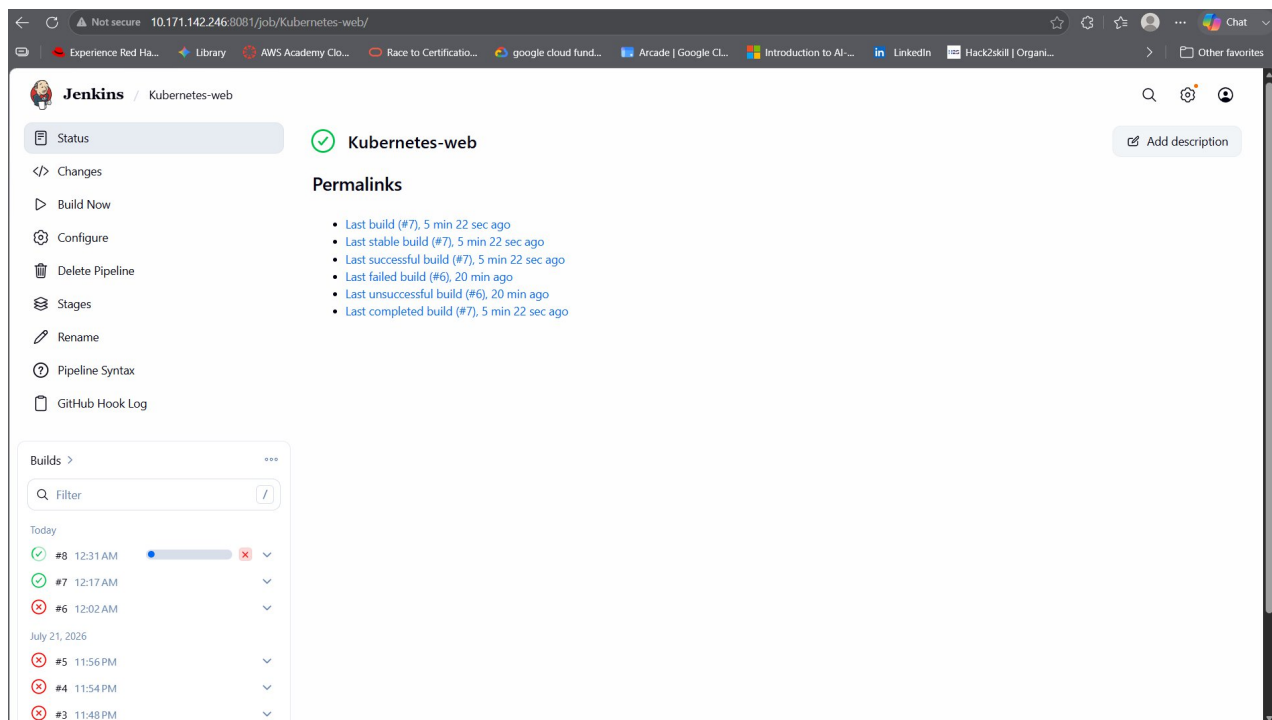
```
[root@localhost kube_jenkins_pipeline]# cat >index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CI/CD Pipeline V2 - Vishal Kayande</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background-color: #f0f2f5; margin: 0; color: #333; }
    header { background-color: #059669; color: white; padding: 3rem 0; text-align: center; }
    .container { max-width: 900px; margin: -2rem auto 2rem; background: white; padding: 2.5rem; border-radius: 12px; box-shadow: 0 10px 15px -3px rgba(0,0,0,0.1); position: relative; }
    h1 { margin: 0; font-size: 2.5rem; letter-spacing: -0.5px; }
    h2 { color: #059669; border-bottom: 2px solid #e2e8f0; padding-bottom: 0.5rem; margin-top: 2rem; }
    .subtitle { font-size: 1rem; opacity: 0.9; margin-top: 0.5rem; font-weight: 300; }
    .architecture-img { width: 100%; border-radius: 8px; margin: 1.5rem 0; border: 1px solid #e2e8f0; }
    .grid { display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem; }
    .card { background: #f8fafc; padding: 1.5rem; border-radius: 8px; border-left: 4px solid #059669; }
    .card h3 { margin-top: 0; color: #0f172a; }
    .success-badge { display: inline-block; background-color: #d1fae5; color: #065f46; padding: 0.25rem 0.75rem; border-radius: 9999px; font-weight: bold; font-size: 0.875rem; margin-top: 1rem; }
  </style>
</head>
<body>
  <div class="container">
    <h1>Kubernetes Jenkins Pipeline</h1>
    <h2>Deployment</h2>
    <div class="grid">
      <div class="card">
        <h3>Deployment</h3>
        <div class="success-badge">Success</div>
      </div>
      <div class="card">
        <h3>Rollout</h3>
        <div class="success-badge">Success</div>
      </div>
    </div>
  </div>
</body>
</html>
```

Pushed the commit to GitHub.

```
[root@localhost kube_jenkins_pipeline]# git add .
[root@localhost kube_jenkins_pipeline]# git commit -m "Updated index.html"
[main ae818b2] Updated index.html
2 files changed, 115 insertions(+), 11 deletions(-)
create mode 100755 kubectl
```

```
[root@localhost kube_jenkins_pipeline]# git push -u origin main --force
Username for 'https://github.com': vishalkayande
Password for 'https://vishalkayande@github.com':
Enumerating objects: 16, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 3 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 26.39 MiB | 478.00 KiB/s, done.
Total 10 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To https://github.com/vishalkayande/Kubernetes-Jenkins-Pipeline.git
+ d7e0df9...ae818b2 main -> main (forced update)
branch 'main' set up to track 'origin/main'.
[root@localhost kube_jenkins_pipeline]#
```

Monitored the Jenkins dashboard as it automatically caught the webhook, initiated Build #8, built the new image, and triggered the Kubernetes rollout.



Kubernetes Cluster Updating the Deployment:


```

root@vishal:~# kubectl get pod -w
NAME                                READY    STATUS    RESTARTS   AGE
my-httpd                            1/1      Running   1 (62m ago) 10h
webapp-deployment-77bdd9c759-55dbj 1/1      Running   0           3m7s
webapp-deployment-77bdd9c759-bkltv 1/1      Running   0           3m7s
webapp-deployment-847c56b56f-2f2x9 0/1      Pending   0           0s
webapp-deployment-847c56b56f-2f2x9 0/1      Pending   0           0s
webapp-deployment-847c56b56f-2f2x9 0/1      ContainerCreating 0           1s
webapp-deployment-847c56b56f-2f2x9 1/1      Running   0           12s
webapp-deployment-77bdd9c759-bkltv 1/1      Terminating 0           20m
webapp-deployment-847c56b56f-f7mrv 0/1      Pending   0           0s
webapp-deployment-847c56b56f-f7mrv 0/1      Pending   0           0s
webapp-deployment-847c56b56f-f7mrv 0/1      ContainerCreating 0           1s
webapp-deployment-77bdd9c759-bkltv 0/1      Terminating 0           20m
webapp-deployment-77bdd9c759-bkltv 0/1      Terminating 0           20m
webapp-deployment-77bdd9c759-bkltv 0/1      Terminating 0           20m
webapp-deployment-847c56b56f-f7mrv 1/1      Running   0           5s
webapp-deployment-77bdd9c759-55dbj 1/1      Terminating 0           21m
webapp-deployment-77bdd9c759-55dbj 0/1      Terminating 0           21m
webapp-deployment-77bdd9c759-55dbj 0/1      Terminating 0           21m
webapp-deployment-77bdd9c759-55dbj 0/1      Terminating 0           21m

```

Refreshed the Kubernetes NodePort URL in the browser, instantly seeing the updated application interface.

Automated CI/CD Pipeline — Version 2.0
Engineered and Deployed by Vishal Kayande
Deogiri Institute of Engineering and Management Studies

✓ Live Container Updated Successfully

Technologies Used

Git & GitHub Jenkins Docker Docker Hub Kubernetes Nginx Linux (VMs)

System Architecture

This live web application is running inside a Kubernetes Pod. Code changes pushed to GitHub automatically trigger a build, containerization, and rollout to the cluster.

```

graph LR
    A[DevOps Engineer] -- "Create Docker File" --> B[Docker File]
    B -- "Commit Code" --> C[Source Code Repo]
    C -- "Pull Code" --> D[Jenkins]
    D -- "Push Image" --> E[Harbor Image Registry]
    E -- "Download Image" --> F[Kubernetes Cluster]
    F -- "Sync" --> G[Update Repo]
    G -- "Pull Changes" --> C

```